Kipu Pay: anonimización en dos pasos con identidad persistente · Design doc

Sebastian Camacho · Reto "Vende los datos sin vender a la gente" · Frontier CDMX 26 · diagrama: `architecture.png`

Problema

Entregar una copia 1:1 del corpus (8.297 registros, ~435k tokens: chats Slack, 26 docs PDF/HTML/MD, 4 tablas) donde ninguna persona natural sea identificable (directa, oculta o indirectamente), con el texto de negocio, las llaves entre tablas y quién habló con quién intactos. Cada persona tiene **una** identidad falsa, la misma en todas las fuentes, en todas las corridas y cuando el corpus crece. Se sirve como API: `/run` < 2 h, `/anonymize` 50 registros < 60 s.

Supuestos (y cómo detecto si son falsos)

- **S1** Las tablas (clientes, empleados, users.json, pqrs) contienen a casi todas las personas: son la semilla del registro. *Detector:* % de menciones sin persona conocida.
- **S2** Los ids internos (CL-, TX-, PQR-, KP-) no son PII; son ancla de identidad y llave de unión (la muestra conserva `CL-88627`). *Detector:* diff con `sample/expected_output`.
- **S3** Los PDF traen capa de texto. *Detector:* body vacío → alerta.
- **S4** Formato de seudónimos y política de generalización (ocupación→sector, barrio→ciudad, año→"hace unos años") se aprenden de la muestra.
- **S5** Crecer = solo agregar. *Detector:* hash por registro viejo antes/después.

Casos difíciles (cada uno es un test)

Dos clientas con el mismo nombre; agente que se llama como una clienta · MAYÚSCULAS sin tildes · correos deletreados ("arroba", "punto") · cédulas con puntos, celulares +57 con espacios · placas, pasaportes, direcciones · metadatos PDF, comentarios y `<meta>` HTML, front matter, nombres de archivo/adjuntos · `<@UID>` · bot de handoff con el nombre del agente; agentes que repiten correo/celular del cliente; terceros · apellido que es lugar · Bancolombia, Carulla, Plaza Botero, Shakira se quedan · barrio + ocupación + año, en el texto **y** en la fila del maestro · 2 docs en inglés.

Fuera de alcance

Reescribir texto con un modelo (pierde parecido, puede inventar) · OCR · servicios/librerías de PII (prohibido) · anonimizar montos, fechas de transacción, ciudades u organizaciones (es la utilidad que paga el comprador) · datos sintéticos.

Arquitectura: primero quién es quién, después cómo se llama

1. **Fetch** (ETag, ≤600 rpm, charset real) y **parse** al esquema normalizado.
2. **Registro de personas** desde las tablas: `personas` (ID `P0001…`, ancla CL-/KP-/uid, tipo, género) · `identificadores` (valor original, normalizado, piezas: parte local del correo, solo dígitos, nombre/apellidos) · `menciones` (registro, campo, texto exacto, persona, cómo se decidió).
3. **Detección y resolución:** formas conocidas con la normalización del juez + regex de formatos + candidatos a nombre. Asignación en cascada: identificador exacto → nombre normalizado con un solo candidato → **Jev** solo si hay duda (noul "¿misma persona?": > 0,85 junta, < 0,15 separa, en medio *persona distinta*; choice persona/org/lugar/marca/figura pública y género). Sin match → persona nueva con el siguiente ID.
4. **Paso A, real → ID:** cada mención se reemplaza por un token `⟦P0042·completo·MAYUS⟧` que guarda la persona y la forma (completo, nombre, apellido, correo, deletreado, celular, cédula…).
5. **Verificación sobre el texto con IDs:** (a) barrido determinista con el `contiene()` del validador: ninguna forma real puede quedar (*gate*); (b) Jev como segunda red: "¿este texto todavía identifica a alguien?". Lo que marque se revisa y entra al registro; nunca aprueba la salida; (c) cuasi-identificadores: generalizar donde la combinación es única en maestro + texto.
6. **Paso B, ID → identidad:** el registro de identidades (`P0042 → Sofía Zambrano`, su correo, celular, cédula…) es *append-only*. Un ID nunca se reasigna; las personas nuevas reciben el siguiente ID y una identidad libre. Cada token se dibuja en la forma que guardó (MAYÚSCULAS sin tildes, deletreado, cédula con puntos).
7. **Barrido final + emitir** los 4 archivos y un `entities.json` honesto (todas las formas encontradas, `kept` con razón).

Decisión	Elegido	Descartado y por qué
Transformación	Dos pasos: real → ID → identidad	Reemplazo directo: mezcla "quién es" con "cómo se llama"; no se puede verificar ni cambiar un seudónimo sin re-detectar
Identidad	ID por ancla estructurada + evidencia cruzada	Nombre como llave: fusiona homónimos
Persistencia	Registro append-only en volumen; respaldo determinista HMAC(sal, ancla) si se pierde	Aleatorio por corrida: rompe el 70 % post-cambio. Hash: es fuga
Seudónimos	Pools colombianos por género, sin ninguna forma real del corpus, 0 tokens compartidos con el original, formatos válidos	Pools libres: un seudónimo igual a otra persona real = fuga
Modelo	Jev con umbrales solo en decisiones dudosas; recibe texto ya con IDs (menos PII afuera); respuestas cacheadas	LLM por registro: caro, no determinista; reglas solas ya dan ~88
Stack/deploy	Node 26 + TS, normalización literal del juez; 1 proceso Railway + volumen	Python: dos normalizaciones divergen. Serverless/túnel: pierde estado
Memoria	`knowledge/` en el repo (reglas, conservados, ocupación→sector), sin PII; registros con originales solo en el volumen	Todo en el repo público: fuga

Plan para medir (antes de construir)

- **M1** Validador oficial OK en muestra y corpus completo: gate de cada deploy.
- **M2** Paso A: 0 formas reales en el texto con IDs; % de menciones por regla vs Jev vs nuevas.
- **M3** Juez local en la muestra (13 tests + 5 públicos): recall de fuga contra las originals del `entities.json` ideal (meta 100 %), conservados intactos (100 %), parecido del texto no personal ≥ 0,98.
- **M4** Entidades vs ideal: fusiones 0, divisiones 0, homónimos separados, "reporta a" intacto.
- **M5** Persistencia: corrida sobre un subconjunto y luego sobre el total → 100 % de IDs e identidades viejas iguales; nuevas únicas.
- **M6** `/anonymize`: conocida → misma identidad; nueva → nueva; homónimos → 2; p95 < 10 s.
- **M7** Costo y tiempo por corrida (< 10 min, llamadas a Jev, cache hit ≥ 95 % en la 2.ª), reportado en `/run/{id}` y `DECISIONS.md`.

Cambio de fuentes y las 3 corridas

Vigilante en la laptop: `/record_index` cada 60 s con `If-None-Match` (304 igual, 200 cambió → hora y delta). **Corrida 1 (60 %) antes de 14:45** con la base por reglas: asegura el 30 % previo. Corrida 2 (75 %) tras confirmar el cambio; corrida 3 (100 %) ≈ 19:15 con el código congelado. No pruebo el entrypoint desplegado mientras evalúan.

Riesgos

Persona no detectada = fuga en todos sus registros → M2 + Jev segunda red · extracción PDF distinta a la ideal → compararla con la muestra en la primera hora · costo/latencia de Jev sin confirmar → fallback por regla.